Benemérita Universidad Autónoma de Puebla

Facultad de ciencias de la computación

Diseño digital

NRC

Proyecto ALU

Nombres de los integrantes

Manuel Montes Pioquinto

Antonio Everardo Morales Paz

Elías Misael Ramírez Baez

Juan Ramon Rangel Velez

Otoño 2025

Introducción

Este proyecto consiste en la construcción de una Unidad Aritmético Lógica (ALU) de 4 bits. El ALU es una de las piezas fundamentales en los procesadores, ya que realiza operaciones aritméticas y lógicas sobre los datos de entrada. En este proyecto, el ALU se ha diseñado utilizando multiplexores y compuertas lógicas lo que permite realizar operaciones como suma, resta, y comparaciones.

Marco teórico

Que es un ALU (Unidad Lógica Aritmética)

Un ALU es un circuito digital combinacional que permite realizar operaciones aritméticas y lógicas básicas sobre datos binarios y representa la parte más pequeña y esencial de un microprocesador. El ALU realiza operaciones como suma, resta, comparaciones y operaciones lógicas como AND, OR, XOR y NOT.

Operaciones Realizadas por la ALU

Operaciones Aritméticas

El ALU realiza operaciones aritméticas fundamentales. las cuales son la suma, la resta, la multiplicación y la división. Para llevar a cabo estas operaciones, esta unidad suele incluir circuitos especializados como sumadores, restadores, multiplicadores y divisores, implementados a través de circuitos lógicos. Todas estas operaciones se realizan sobre números binarios y se utiliza el complemento a dos para los números negativos.

Operaciones Lógicas

Se encarga de realizar operaciones lógicas a nivel de bits como AND incluyendo su acarreo, OR, NOT, XOR, NAND y NOR. Al igual que la unidad aritmética, la unidad lógica se implementa utilizando puertas lógicas básicas

Operaciones de Comparación

El ALU también es capaz de realizar operaciones de comparación como igualdad, desigualdad, mayor que, menor que. Estas se realizan restando los números y analizando el resultado.

Control y Estado de la ALU

El funcionamiento de la ALU está regido por señales de control y proporciona información sobre el resultado de las operaciones

Selección de Operaciones

El opcode es un código binario que especifica la operación aritmética o lógica deseada. La unidad de control de la ALU decodifica el opcode para determinar la operación específica. Las señales de control seleccionan la operación aritmética o lógica apropiada dentro de la ALU utilizando multiplexores.

Arquitecturas del ALU de 4 bits

ALU con Multiplexores y Puertas Lógicas

Implica el uso de multiplexores para seleccionar la operación a realizar y la implementación de cada operación utilizando combinaciones de puertas lógicas básicas como AND, OR, NOT y XOR junto con el manejo del acarreo entre los bits.

Elaboración del ALU

Materiales para la practica

Los materiales utilizados para la realización de este proyecto son:

- 2 Dip-Switches
- 17 resistencias de 300 ohms
- 5 LED
- 4 protoboard
- 2 multiplexores 74LS157 (cuádruple de 2 a 1)
- 2 multiplexores 74LS153 (cuádruple de 4 a 1)
- 4 compuertas XOR 74LS86
- 1 comparador 74LS85
- 3 compuertas OR 74LS32
- 3 compuertas AND 74LS08
- 1 compuerta NOT 74LS04

Datasheet

Multiplexor 74LS157

Entradas

S: Entrada común de selección

E: Habilitación. Activa bajo nivel.

I0a-I0d: Entradas de datos desde la fuente 0

I1a-I1d: Entradas de datos desde la fuente 1

Salidas

Za-Zd: Salidas del multiplexor.

Alimentación

Vcc: Pin 16 entrada de alimentación de 5V.

Tierra

GND: Pin 8 conexión a tierra.

Diagrama de conexión

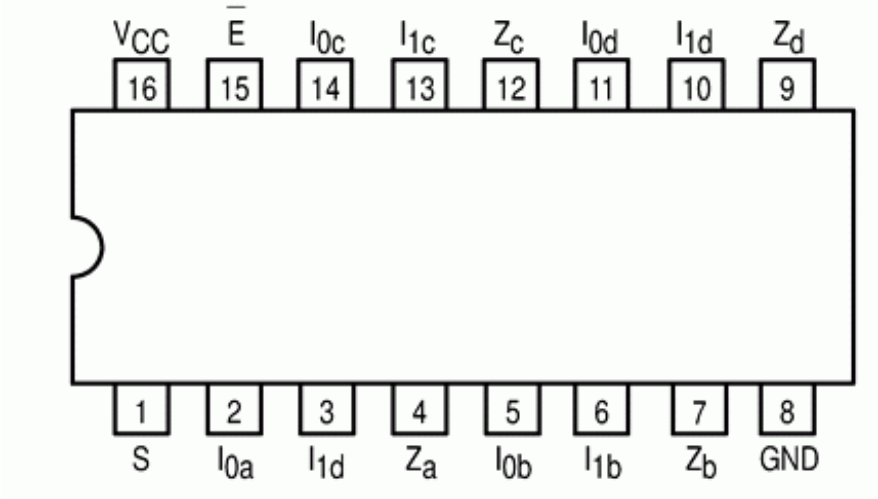


Ilustración 1 Diagrama de conexión 74LS157

Multiplexor 74LS153

Entradas

S0, S1: Son las entradas de selección para elegir entre las líneas de entrada.

E : Entrada de habilitación. Cuando está en LOW, activa el multiplexor, y cuando está en HIGH, lo desactiva.

I0a-I0d: Son las 4 entradas de datos del primer grupo.

I1a-I1d: Son las 4 entradas de datos del segundo grupo.

Salidas

Za, Zb: Son las dos salidas del multiplexor.

Alimentación

Vcc: Pin 16 entrada de alimentación de 5V.

Tierra

GND: Pin 8 conexión a tierra.

Data inputs

Este multiplexor usa otros dos multiplexores independientes, por lo que cada uno cuenta con 4 entradas.

Diagrama de conexión

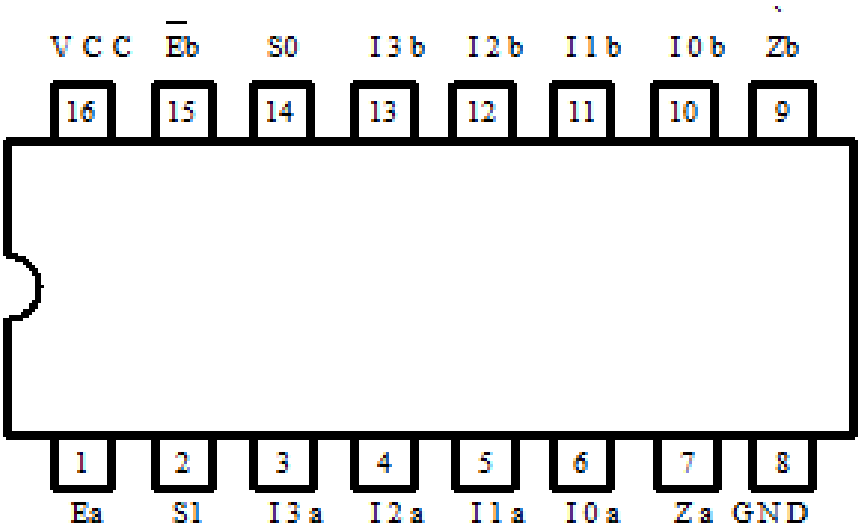


Ilustración 2 Diagrama de conexión 74LS153

Comparador 74LS85

Entradas

A0–A3, B0–B3*:* Entradas de datos

IA>B, IA<B, IA=B: Para conexión en cascada.

Salidas

OA>B, OA<B, OA=B

Alimentación

Vcc: Pin 16 alimentación de 5V.

Tierra

GND: Pin 8 conexión a tierra.

Diagrama de conexión

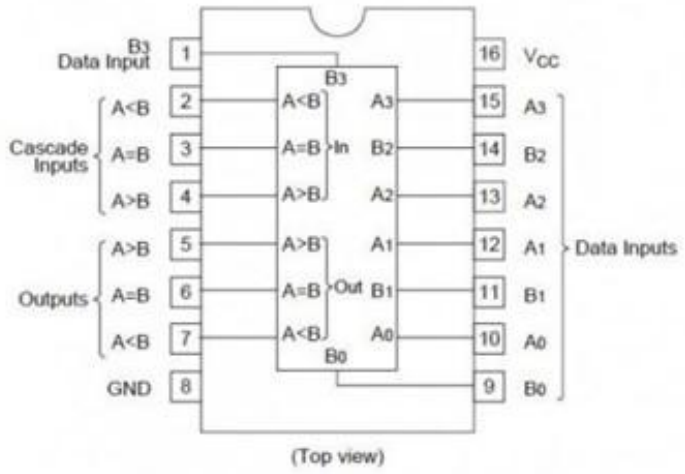


Ilustración 3 Diagrama de conexión 74LS85

Diseño digital en Logisim

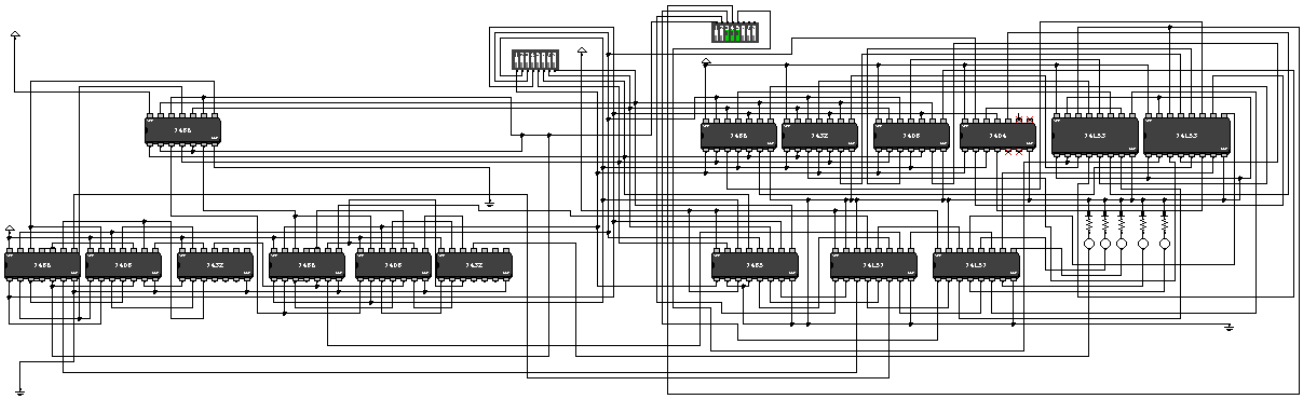


Ilustración 4 Diseño digital del ALU en Logisim

Suma

Para el desarrollo de la suma en el proyecto haremos uso primeramente de un sumador Full Adder el cual adicionalmente de la suma también nos permite manejar sumas de mas de 2 bits.

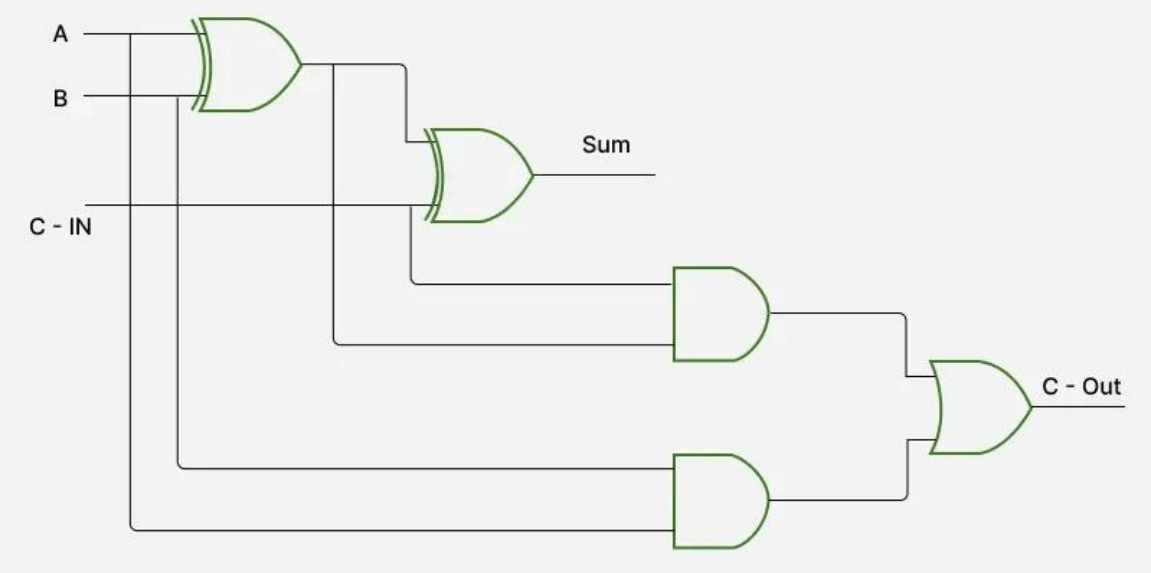


Ilustración 5 Sumador Full adder

Para realizar este sumador hacemos uso de el XOR el cual actúa como un sumador simple de un solo bit de forma que no pase la señal en cado de que este se 1 en A y 1 B. Seguido de esto se usa un AND conectado a las entradas de forma que si la señal es de 1 + 1 el XOR devuelva 0 mientras que el AND devuelve el carry de 1 formando así un sumador half adder.

Para que el ALU sea capas de hacer sumas de dos números de mas de dos bits es necesario añadir un segundo half adder que nos permita sumar el resultado de la primera suma con el acarreo de entrada y finalmente se añade un OR para dar el resultado de los dos acarreos de modo que terminamos con un sumador Full adder compuesto por dos half adder el cual funciona de la siguiente manera.

Primer Half-Adder:

- Suma de A y B.
- Genera Sum1 y Carry1.

Segundo Half-Adder:

- Suma de Sum1 con el Carry-in.
- Genera Sum y Carry2.

Combinación del carry:

- El Carry-out final (Cout) es el resultado de la OR de Carry1 y Carry2.

Tabla de verdad

A	B	Cin	Sum (A ⊕ B ⊕ Cin)	Cout ((A · B) + (Cin · (A ⊕ B)))
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Diagramas K

Diagrama de Sum

A \ B, cin	00	01	11	10
0	0	1	0	1
0	1	0	1	0

SUM = A ⊕ B ⊕ Cin

Diagrama de Cout

A \ B, cin	00	01	11	10
0	0	0	1	0
0	0	1	1	1

$Cout = (A \cdot B) + (Cin \cdot (A \oplus B))$

Resta

Para realizar la resta debemos transformar un numero binario en su complemento a 2, para lo cual se invierte el número y se le suma 1. En el caso de la resta ignoraremos el carry y recordemos que el bit mas significativo indica si el número es negativo con un 1 y con un 0 si es positivo, por lo tanto, ahora tenemos 7 dígitos positivos, 8 negativos y un cero, como se muestra en la siguiente tabla.

Decimal	Binario	Complemento a 2
-8	----	1000
-7	----	1001
-6	----	1010
-5	----	1011
-4	----	1100
-3	----	1101
-2	----	1110
-1	----	1111
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111

Para llevar acabo estas operaciones en un circuito primero invertimos la entrada B haciendo uso de el XOR con una entrado que contenga un 1 lógico, lo cual ocasiona que actúe como un NOT.

X	Y	XY' + X'Y (XOR)
0	1	1
1	1	0

Para el segundo paso utilizamos el primer carry de entrada Simulado que estamos sumando 1, para lo cual un botón se conecta directo al XOR y al primer carry de entrada haciendo que nuestra suma se convierta ahora en una resta.

Para esto el interruptor 1 del segundo dip switch se utiliza como 1 lógico en caso de que se quiera realizar una resta debido a que esta directamente conectado a una pata del XOR y al primer carry.

Implementación en el ALU

Operaciones Aritméticas

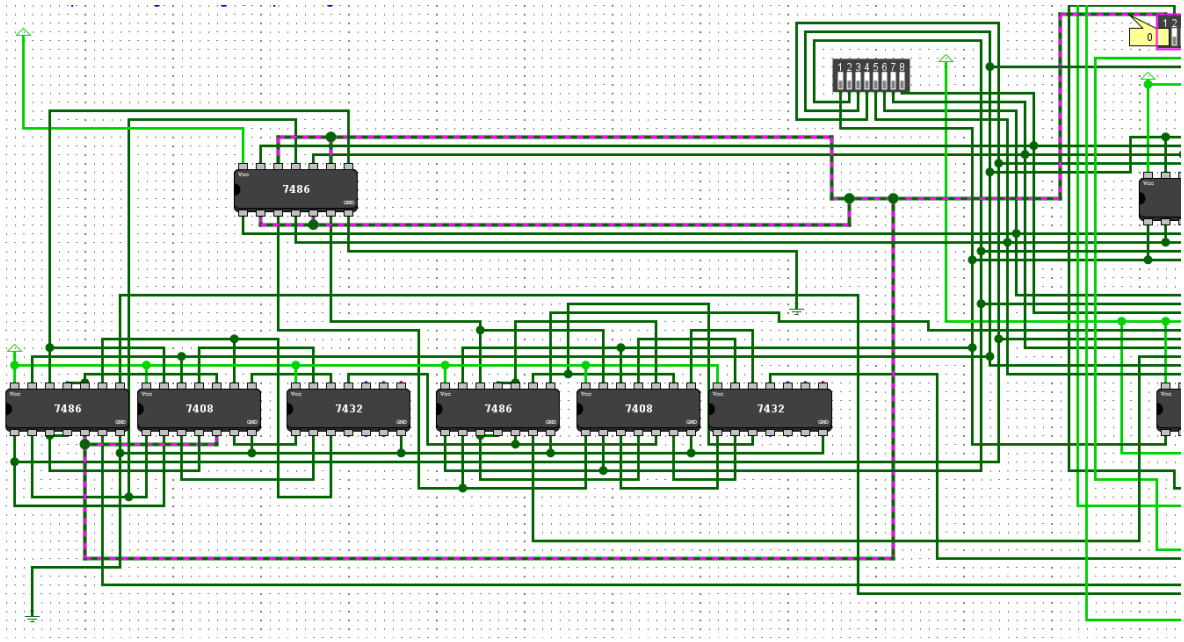


Ilustración 6 Sección izquierda del ALU

Para realizar las conexiones necesarias para implementar tanto la suma como la resta implementamos un total de 4 full adder conectados lo cual nos daría un total de 8 AND, 8 XOR y 4 OR, por cuestión de comodidad decidimos usar 2 CI AND, 2 CI XOR y 2 CI OR permitiéndonos de esta manera tener un mayor orden en el diseño del circuito.

Se conectaron los CI en paralelo en la siguiente disposición:

Un 74LS86 seguido de un 74LS08 y por último un 74LS32

En cada una de las dos sucesiones de CI se utilizan tanto las entradas y salida de los CI siendo correspondiente a la parte baja de la primera sucesión de CI la primera suma de bits la parte

alta la segunda suma de bits y de igual manera con la segunda sucesión de CI la parte baja representa la tercera suma de bits y la parte alta la curta suma de bits.

Antes de que las entradas de B lleguen a la suma estas pasan por el 74LS86 el cual cuenta con una entrada directamente conectada al dip switch permitiéndonos invertir el valor para realizar las restas.

Operaciones lógicas

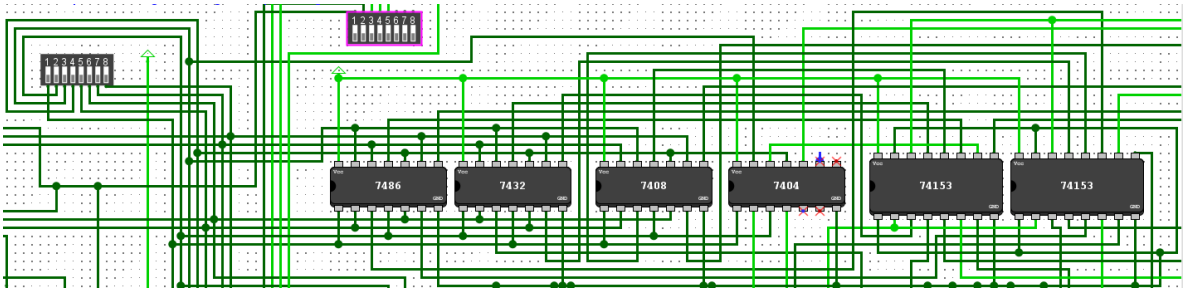


Ilustración 7 Sección derecha del ALU

Esta sección del circuito de encarga de realizar las operaciones lógicas NOT, XOR, AND y OR según la combinación de entradas.

Para el NOT únicamente se invierte la entrada A posterior mente se combinan la entrada A y B resultando así en una salida C la cual se conecta a uno de los multiplexores 74LS153.

Cada circuito integrado tiene como mínimo 4 compuertas lógicas con dos entradas, a excepción del 74LS04 que tiene 6, sin embargo, únicamente le daremos uso a 4 de estas y se conectaran de la siguiente manera:

Pin 1 y 2 → bit 4 de A y B → salida en Pin 3 → a entrada 2C1 del segundo 74LS153

Pin 4 y 5 → bit 3 de A y B → salida en Pin 6 → a entrada 1C1 del segundo 74LS153

Pin 9 y 10 → bit 1 de A y B → salida en Pin 8 → a entrada 1C1 del primer 74LS153

Pin 12 y 13 → bit 2 de A y B → salida en Pin 11 → a entrada 2C1 del primer 74LS153

Conexión del 74LS153.

El primer multiplexor refleja el resultado de los primeros dos bits y el segundo representa los siguientes dos bits.

Los Strobes A y B, de ambos multiplexores van directamente a tierra, para mantener siempre activos estos, S1 de ambos 74LS153 se conectan entre si al igual que los S0. Las entradas restantes van conectadas a una compuerta lógica las cuales se repiten para ambos multiplexores. Las conexiones se ven reflejadas de la siguiente manera:

1C0 y 2C0 de ambos multiplexores se conectan a las salidas del 74LS04

1C1 y 2C1 de ambos multiplexores se conectan a las salidas del 74LS86

1C2 y 2C2 de ambos multiplexores se conectan a las salidas del 74LS08

1C3 y 2C3 de ambos multiplexores se conectan a las salidas del 73LS32

La razón por la que nunca se apaga los multiplexores es porque los circuitos a diferencia de las computadoras se pueden manejar de manera concurrente, haciendo varios procesos a la vez, lo que usamos para que sea más rápido el cálculo.

Combinacional Lógica aritmética

Comparación

Para la comparación se utilizó un 74LS85 que es un comparador de dos números de 4 bits.

Para realizar las comparaciones se utilizan las siguientes formulas

$A == B \rightarrow F = (A\ xor\ B)'$

$A > B \rightarrow F = AB'$

$A < B \rightarrow F = A'B$

O lo que es igual a:

$F = A > b + A < B + A == B$

A	B	A<B (A'B)	A==B (AXORB)'	A>B (AB')
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

En base a esto para describir las fórmulas de más de 1 bit, se debe considerar que si se tiene un numero 1000 y 0111, el método mejor para identificar al mayor es comprar el bit mas grande por lo que.

A==B -> F = $\prod_{i=0}^{n} (A \oplus B)'$

Se mantiene para cualquier cantidad de bits

Sin embarga par el resto de los operadores primero debemos verificar si A > B, después se verifica si A = B, de así serlo pasamos a los siguientes bits A2 = B2 y de esta forma avanzamos en estos sucesivamente dando como resultado:

A > B -> $\sum_{i=0}^{n-1}(\prod_{j=0}^{i-1}(A_{n-1-j} \oplus B_{n-1-j})' * A_{n-1-i} * B_{n-1-i}{}')$

Extendido para 4 bits

A > B = (A3·B3) + (A3⊙B3) · A2·B2 + (A3⊙B3) (A2⊙B2) · A1·B1 + (A3⊙B3) (A2 ⊙B2) (A1⊙B1) · A0·B0

A<B -> $\sum_{i=0}^{n-1}(\prod_{j=0}^{i-1}(A_{n-1-j} \oplus B_{n-1-j})' * A_{n-1-i}{}' * B_{n-1-i})$

Otro método de comparación puede ser por medio de la resta, restamos A – B si el resultado es negativo entonces A < B, si es positivo A > B y si es 0 A == B

Para esto primero se deben activar las restas junto con las comparaciones y tener en cuenta que el bit mas significativo en la resta indica si el resultado es positivo o negativo, entonces

reconduciremos el 4to bit, a la entrada B1 del multiplexor. Después, necesitamos saber si el resultado es cero para ello tenemos la siguiente tabla de verdad.

x	y	z	w	F1
0	0	0	0	1
0	0	0	1	0

De esta forma el circuito nos dirá si la resta es 0, de forma que solo falta saber cuando A > B y cuando A == B por lo que necesitamos de las funciones:

A > B	A = B	F2	F3 A = B
0	0	0	0
0	1	0	1
1	0	1	0
1	1	0	1

F2

X/Y	0	1
0	0	0
1	1	0

F2 = XY’

F3

X/Y	0	1
0	0	1
1	0	1

F3 = Y

Con esto hecho podemos completar el circuito incluyendo también el comprador.

Por último, se usan dos multiplexores 75LS157 de 2 a 1, para separar la parte lógica de la parte aritmética, y la suma/resta de la comparación.

El primer 74LS157 decide si queremos compara o sumar las entradas B tienen las sumas y las entradas A el resultado de la suma, el resultado se conecta con las entradas A del segundo 74LS157 y en las entradas B del segundo 74LS157 son las salidas de los multiplexores 74LS153. Por ultimo las salidas del segundo 74LS157 se conectan a leds de color azul y el primer bit se conecta al carry de la última suma.

De forma que:

El primer 74LS157 selecciona si se va a realizar una suma o una comparación. Si se selecciona suma, los resultados de los Full Adders se envían al segundo 74LS157. El segundo 74LS157 decide cuál de los resultados (de suma o comparación) se pasa a la salida. Los resultados finales se muestran en los LEDs.

Con esto finalizamos el circuito.

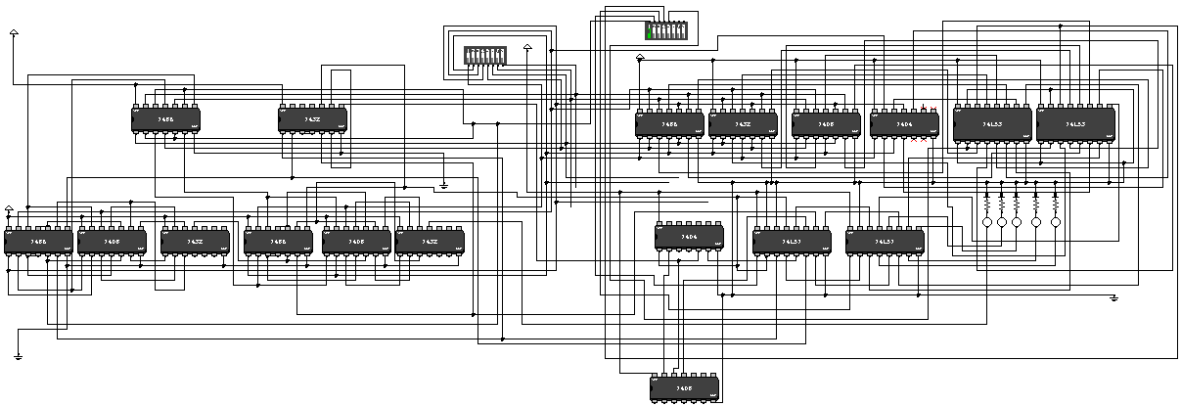


Ilustración 8 Circuito ALU completo

Funcionamiento

1er dipSwitch

Primer numero				Segundo numero			
1	2	3	4	5	6	7	8
2^3	2^2	2^1	2^0	2^3	2^2	2^1	2^0

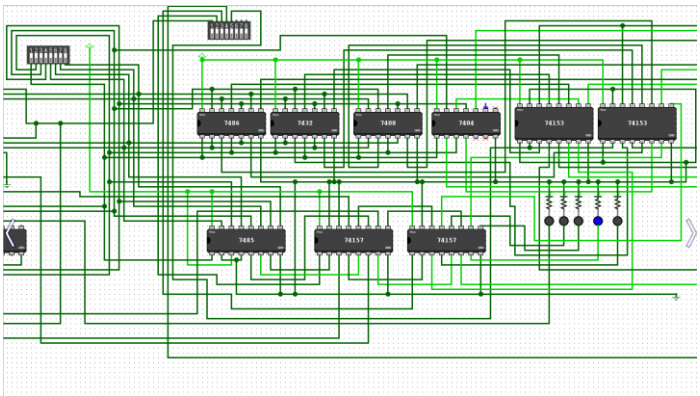
2do dipswitch

1	2	3	4	5	Opción
Suma/Resta	comparación/Suma	aritmético/Lógica	0	0	Negación de A
			0	1	AND
			1	0	XOR
			1	1	OR

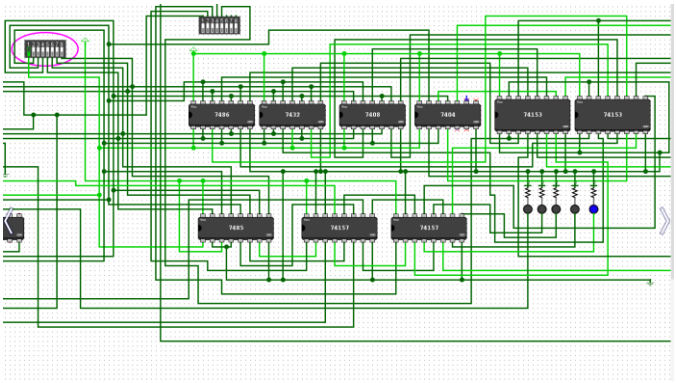
Ejemplos de funcionamiento

Comparacion

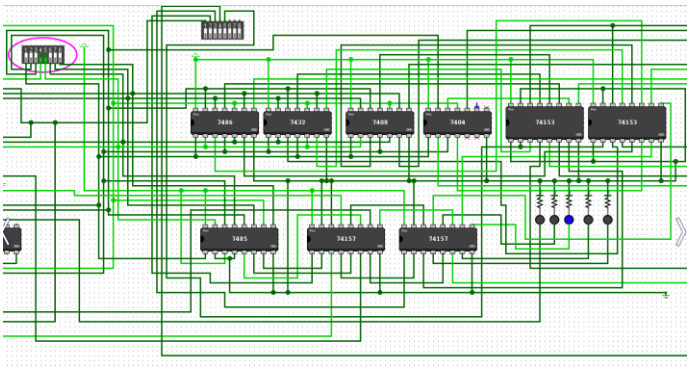
A == B



A>B

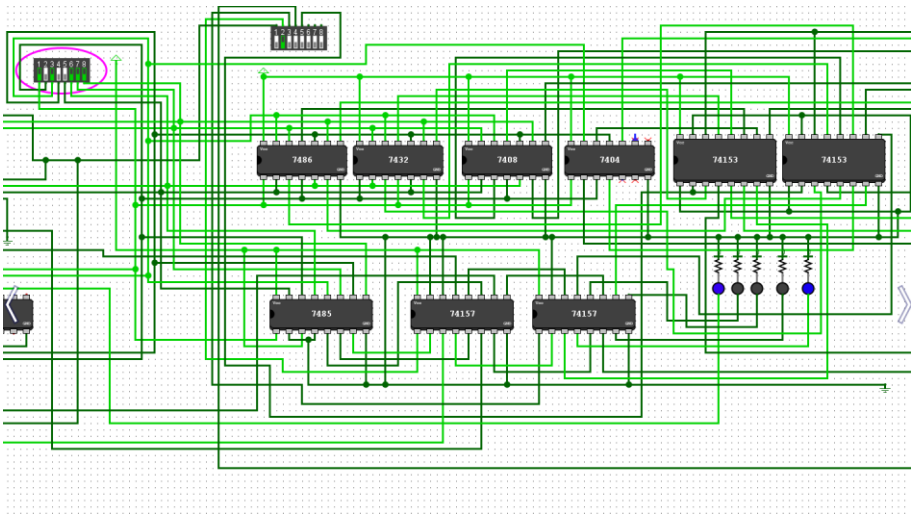


A < B



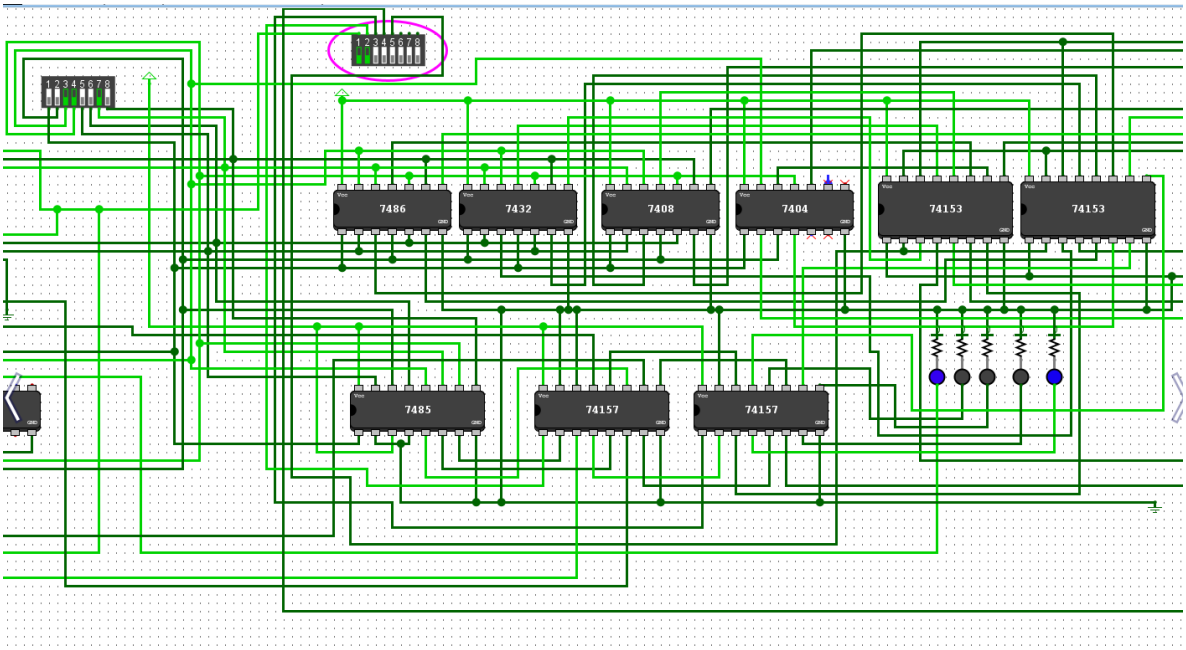
Suma

1010 + 0111 = 10001

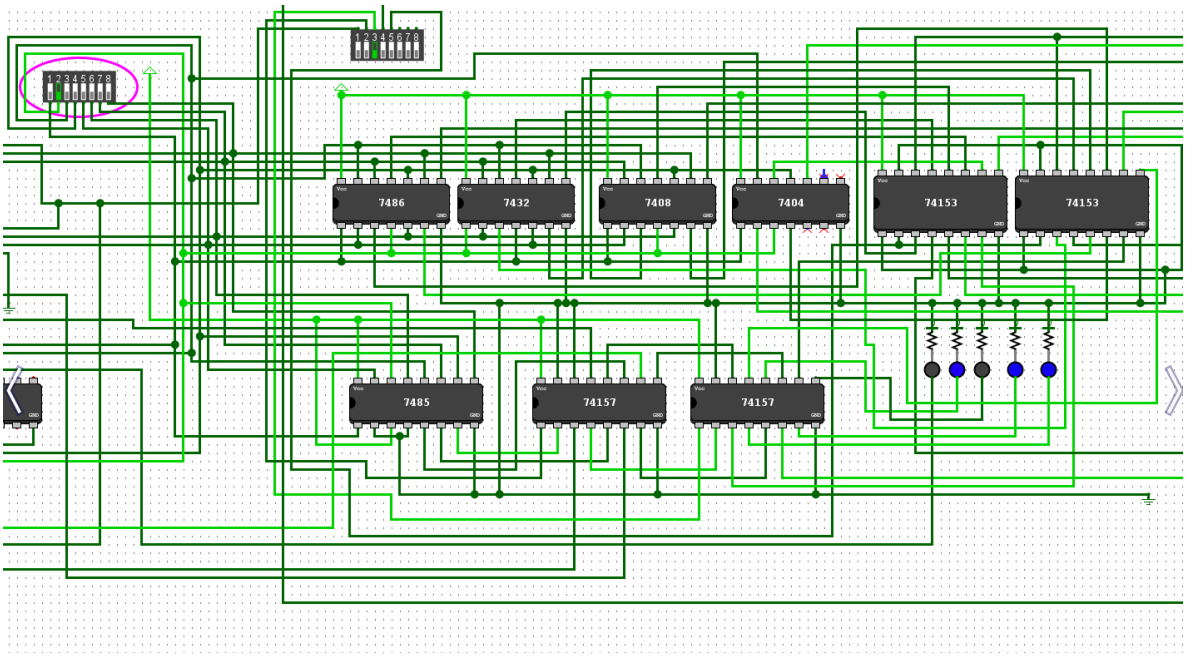


Resta

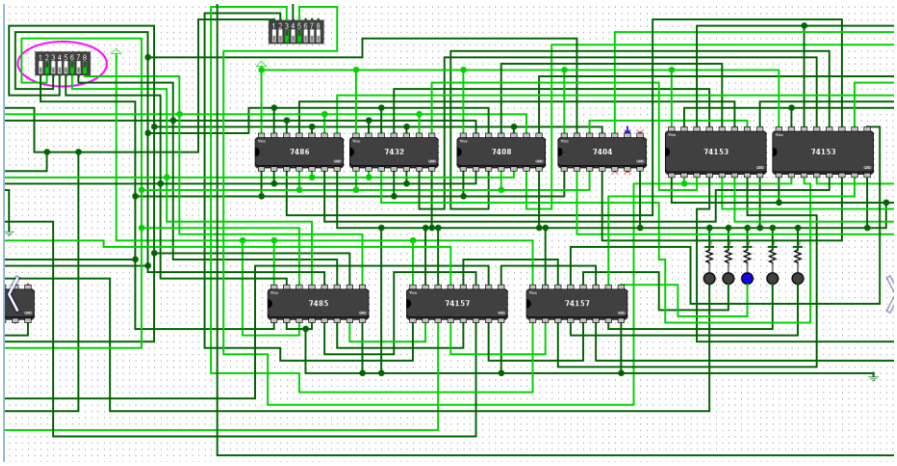
0011(3) – 0010 (2) = 0001



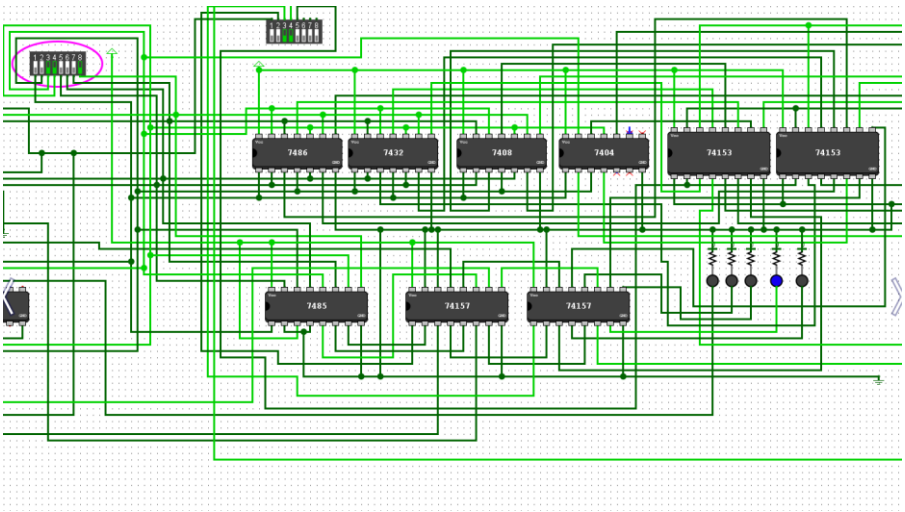
Not



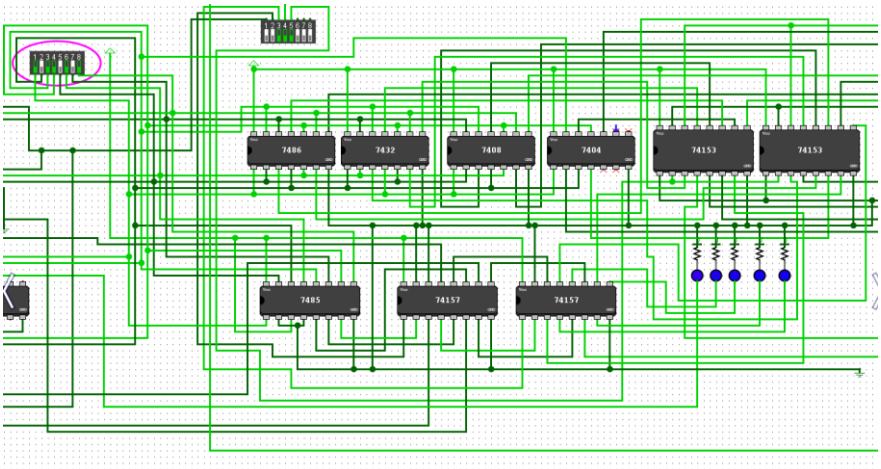
AND


7486
7432
7408
7404
74153
74153
7485
74157
74157

XOR


7486
7432
7408
7404
74153
74153
7485
74157
74157

OR


7486
7432
7408
7404
74153
74153
7485
74157
74157

Referencias

National University of Singapore. (s.f.). *Control Signal*. Obtenido de National University of Singapore Web site: https://www.comp.nus.edu.sg/~adi-yoga/CS2100/ch08c1/

CircuitVerse. (2024). *Arithmetic Logic Unit*. Obtenido de Learn CircuitVerse : https://learn.circuitverse.org/docs/comb-lsi/alu.html

Fairchild Semiconductor. (s.f.). *74LS85 Datasheet* . Obtenido de Alldatasheet Web site: https://www.alldatasheet.es/datasheet-pdf/pdf/51093/FAIRCHILD/74LS85.html

Florida International University. (s.f.). *CDA-4101 Lecture 16 Notes.* Obtenido de FUI web site: https://users.cs.fiu.edu/~prabakar/cda4101/Common/notes/lecture16.html

Geeksforgeeks. (28 de Diciembre de 2024). *Introducción de ALU y ruta de datos.* Obtenido de https://www.geeksforgeeks.org/introduction-of-alu-and-data-path/

Johann F. Pachón, M. M. (2010). *SIMULADOR DIDÁCTICO DE UN PROCESADOR PCSIM2000.* Santafé de Bogotá D.C., Colombia: Pontificia Universidad Javeriana.

Lenovo. (s.f.). *Arithmetic Logic Unit.* Obtenido de Lenovo Web site: https://www.lenovo.com/es/es/glossary/arithmetic-logic-unit/

Lenovo. (s.f.). *Control Unit.* Obtenido de Lenovo Web site: https://www.lenovo.com/es/es/glossary/control-unit/

Motorola, Inc. (s.f.). *74LS153 Datasheet.* Obtenido de Alldatasheet Web site: https://www.alldatasheet.es/html-pdf/5662/MOTOROLA/74LS153/258/1/74LS153.html

Motorola, Inc. (s.f.). *SN74LS157 Datasheet.* Obtenido de Alldatasheet Web site: https://www.alldatasheet.es/datasheet-pdf/pdf/5664/MOTOROLA/SN74LS157.html

National University of Singapore. (s.f.). *ALU Control*. Obtenido de National University of Singapore Web site: https://www.comp.nus.edu.sg/~adi-yoga/CS2100/ch08c2/

NortonVPN. (s.f.). *Arithmetic logic unit.* Obtenido de NortonVPN Web site: https://nordvpn.com/es-mx/cybersecurity/glossary/arithmetic-logic-unit/

Óscar Robles, Á. S. (s.f.). *Estructura y Tecnología de Computadores (ITIG).* Obtenido de Univercidad Rey Juan Caarlos: https://d1wqtxts1xzle7.cloudfront.net/33225342/tema13_alu_numeros_enteros-libre.pdf?1394891350=&response-content-disposition=inline%3B+filename%3Dtema13_alu_numeros_enteros.pdf&Expires=1746677893&Signature=C8ZSmxc0HLkAGxKMbEtOQu5lejBuLEavthyfeIzifxTl5Q~-h

Real Digital. (s.f.). *Arithmetic Logic Units (ALU)*. Obtenido de Real Digital Web site: https://www.realdigital.org/doc/5b8eed1f35370daee2489d692a29649b

Real Digital. (s.f.). *Arithmetic Logic Units (ALU)*. Obtenido de Real Digital Web site: https://www.realdigital.org/doc/5b8eed1f35370daee2489d692a29649b

Schmalz, M. (s.f.). *Organization of Computer Systems:.* Obtenido de Uneversity of Florida WEb site: https://www.cise.ufl.edu/~mssz/CompOrg/CDA-proc.html

Toomey, W. (02 de Mayo de 2011). *Week 2 Tutorial - Building an ALU*. Obtenido de Minnie Web site: https://minnie.tuhs.org/CompArch/Tutes/week02.html

University of Minnesota. (s.f.). *execution activities.* Obtenido de University of Minnesota Web site: https://www.d.umn.edu/~gshute/mips/execution-activities.pdf

UTAH. (s.f.). *Design of the ALU.* Obtenido de https://my.ece.utah.edu/~kalla/ECE3710/lab1-11.pdf